Centro Architecture & Philosophy

This document is the single source of truth for Centro's architectural principles and product philosophy. Every design and implementation decision in this codebase is expected to follow it. It is version-controlled alongside the code it governs, updated at the end of every roadmap milestone, and rendered to `reports/Centro-Architecture-and-Philosophy.pdf` for distribution outside the repo.

When an implementation decision conflicts with this document, the document wins unless the conflict is explicitly raised, discussed, and resolved here first — never resolved silently in code.

Chapter 1 — Learning First

Every client begins in Learning Mode.

Centro does not require a perfect setup before work begins. Onboarding exists to provide enough information to start working safely, not to perfectly configure every client.

Excel imports, onboarding choices, and existing records create only an Initial Client Profile. From the very first collection cycle, Centro observes reality, learns from recurring behaviour, and continuously improves the profile.

Centro does not guess. It observes, suggests, confirms, and only then learns.

Core principles:

- Start working immediately.
- Learn continuously.
- Never guess when confidence is low.
- Improve from real behaviour, not assumptions.

Chapter 2 — The First Collection Cycle

Every client enters the first month in Learning Mode.

Centro requests the client's documents, waits until the client confirms that the submission is complete, and then treats the collection as one business event to learn from.

The first cycle is not a validation phase. It is a learning phase designed to understand how the client actually works — specifically, which documents that client actually sends, nothing broader.

Chapter 3 — Observe → Suggest → Confirm → Learn

Every meaningful change to a client's document collection profile follows the same lifecycle:

1. **Observe.**
2. **Suggest.**
3. **Confirm** with the client whenever appropriate, through WhatsApp.
4. **Learn.**

Centro never changes a client's profile because of a single event or assumption. A pattern must repeat before Centro treats it as worth asking about, and even then, only the client's explicit confirmation — never the pattern alone — changes the profile.

Chapter 4 — Learning Without Becoming a Burden

Learning must never create unnecessary work.

Centro should quietly observe whenever possible. It asks questions only if the answer has long-term value and the benefit outweighs the interruption.

Questions must be short, natural, and asked only once — if a client has already declined a suggestion, Centro does not ask about the same thing again.

The accountant reviews exceptions — not routine learning.

Core principle: Centro becomes smarter while becoming quieter.

Chapter 5 — The Living Client Profile

Client profiles are living profiles.

Knowledge comes from Excel, onboarding, collected documents, recurring patterns, and confirmed client responses.

No single source is absolute truth.

Businesses evolve, so the client's document collection profile evolves with them — see Chapter 8 for the exact, permanent boundary of what "evolves" is allowed to mean.

Chapter 6 — Decision Hierarchy

Every decision follows the same hierarchy, most specific and most certain signal first:

1. **Learned Knowledge** — a specific fact this organization has already confirmed about its own clients or data. Checked first because it is the most reliable signal available: it was explicitly confirmed, not inferred, and it is scoped to exactly this organization.
2. **Business Rules** — deterministic, universal rules that apply the same way across every organization (e.g. recognizing standard terminology, hard confidence floors, required-confirmation gates). Used whenever no more specific learned fact exists for this organization yet.
3. **Artificial Intelligence** — a supporting tool, reached only once the above two layers find nothing with sufficient confidence. AI is never the foundation of a decision in Centro.
4. **Human Review** — whenever confidence remains insufficient after all three layers above, a person decides. Centro never guesses.

A previously confirmed, organization-specific fact always outranks a generic rule. This is a deliberate refinement of "business rules first": a rule that is merely generic is less trustworthy than a fact Centro was explicitly told and is now honoring. Business Rules remain the universal floor beneath Learned Knowledge — every organization gets at least this level of correctness even before it has taught Centro anything of its own — but never above a fact that organization has already confirmed.

Chapter 7 — Invisible Automation

The best automation is the one users barely notice.

Routine work happens automatically.

Learning happens continuously.

Notifications are meaningful.

Every interruption must have value.

Success is measured by how much work Centro completes without demanding attention.

Chapter 8 — The Boundary of Learning

Centro learns exactly one thing: **which documents should be requested from each client.**

This is refined only through observed recurrence and explicit client confirmation, following the Chapter 3 lifecycle exactly — never inferred, never assumed, and never applied from a single occurrence.

Centro does not learn, infer, or adapt:

- The client's underlying business, workflow, or operations.
- The accounting firm's own office policies — including business hours, working days, reminder cadence, or the day a collection period begins.

These remain office policy: configured explicitly by the accountant, once, and changed only by the accountant. Centro never modifies them automatically, regardless of what it observes.

This boundary is permanent. It applies to every learning mechanism in Centro — present and future — not only document collection. A future milestone that would cause Centro to learn or adapt timing, scheduling, or any office policy is out of scope by definition, not a case-by-case judgment call.

Extension (Product Evolution M7): the boundary now has a second, absolute form. For a `one_time`-workflow organization (Chapter 9), Centro does not learn *at all* — not a client's document profile, not a recurring pattern, nothing. This is stronger than "refined only through observed recurrence": it is "never even observed." The guarantee is enforced inside the three functions that ever write learned state (`recordAdHocDocumentObservation`, `detectMissingRequirements`, `recordLearnedDocumentPattern`), each checking `isOneTimeWorkflowOrganization(organizationId)` as their first line — not only by Workflow B's UI never exposing the paths that would trigger it. A future caller that forgets this check still cannot cause Workflow B to learn.

Chapter 9 — The Two-Workflow Product Model

Everything in Chapters 1–8 describes Centro as built for one kind of business: recurring, document-collecting professional services (the accounting-firm flagship). The Product Evolution roadmap generalized Centro into a platform for *any* document-collecting business, without rewriting that flagship experience. It did this by introducing exactly one new, permanent fork, decided once at onboarding and never silently changed afterward.

9.1 Two Facts, Set Once

Every organization declares two things during onboarding that no chapter before this one assumed varied:

- `organizations.businessCategory` — what kind of business the office itself is (accountant, tax advisor, lawyer, real estate, mortgage advisor, insurance, HR, finance, or a free-text "other").

This is deliberately distinct from the pre-existing `business_types` table, which classifies the office's *clients* and is a Workflow-A-only concept. Category is used for onboarding personalization and AI-driven document suggestions — never to change how the learning engine itself behaves.

- **`organizations.workflowType`** — `recurring` (Chapters 1–8, exactly as written) or `one_time` (this chapter). Every organization that existed before this roadmap defaults to `recurring`, so nothing in Chapters 1–8 changed for a single existing organization the day this shipped.

Both are meant to be permanent, set-once onboarding decisions — not a Settings toggle a user can flip later. This is enforced structurally, not just by omission: once `organizations.onboardingCompletedAt` is set, the onboarding wizard itself becomes unreachable (Product Evolution M8) — a completed organization visiting `/onboarding` directly is redirected straight back to its dashboard before any onboarding action, including a workflow-type change, can run. Switching workflows remains possible only as a deliberate, out-of-band data-migration operation, never as a page a logged-in user can stumble back into.

9.2 Workflow A (Recurring) — Unchanged, Now Generalized at the Edges

For an `accountant` or `tax_advisor` organization, Workflow A is byte-identical to Chapters 1–8: the same wizard, the same Excel analysis, the same starter business types, the same learning engine. For every other declared category, only the onboarding *starting point* generalizes — a mocked-AI module (`src/lib/ai/businessCategorySuggestions.ts`) supplies category-appropriate starter client types and document checklists instead of the five hardcoded Israeli accounting types, falling back to keyword-matching and then a broad, never-empty "General Client" starter for a genuinely novel free-text category. Once seeded, a business type behaves identically regardless of which tier produced it — Chapter 3's lifecycle, Chapter 6's Decision Hierarchy, and the learning engine itself never change based on category. Only the starting point does.

9.3 Workflow B (One-Time) — A Different, Smaller, Non-Learning Product

A `one_time`-workflow organization gets a materially different experience from the moment onboarding forks (after the shared Welcome → Office Info → Business Type → Collection Style steps): an optional client import that stores only name and phone (no classification, no analysis), then Working Hours only (no reminder cadence or collection-day-of-month — office-policy concepts with no meaning for a one-off request), then its own summary and completion screen. It lands on its own dashboard and navigation, built around one new primary surface:

A Template is a bare `services` row belonging to a `one_time`-workflow organization — no new table. A Template has a name, a list of required documents (AI- suggested per business category via `suggestTemplateLibrary`, fully user-editable — add, rename, reorder, remove), and any number of assigned clients (`client_services`, the same join table Workflow A's service assignment already used). Sending a Template creates one ordinary `collectionRequest` per assigned client — reusing `snapshotServiceRequirements` unchanged — and delivers it either immediately or at a scheduled future time via the single shared `attemptScheduledDelivery` function (`src/lib/scheduledSend.ts`);

there is no separate "send now" code path. From the moment of delivery onward, Workflow B's collection requests flow through the *exact same* shared infrastructure as Workflow A's: WhatsApp messaging, OCR/classification (Chapter 6), Google Drive's one-folder-per-client organization, document tracking, and the reminder engine (Chapter 7) — all reused without modification.

The one thing Workflow B never does, by design, is learn (see Chapter 8's extension above). Templates are entirely explicit and user-edited; there is no per-client document-profile history to observe, because a one-time office's relationship with a client is, definitionally, one-time.

9.4 The Permanent Distinction, In One Sentence

Recurring Workflow: Centro learns document requirements over time, per client. **One-Time Workflow:** Centro never learns; the user manages reusable Templates instead. Every other product decision in this chapter follows from that one line.

9.5 UX Polish — Terminology & Surfaces

A dedicated UX-only round (no architecture, no business logic) brought the product's own terminology in line with the two-workflow model this chapter describes, and closed two onboarding-era gaps:

- **"Office" → "Business."** The office's own profile (name, logo, the Settings section that edits them) now reads "Business" throughout — the office/firm-specific framing didn't fit a platform meant for lawyers, HR teams, and mortgage advisors as much as it fit the accounting flagship. Client-facing automated copy (the WhatsApp greeting) and public marketing pages were deliberately left untouched — different audience, out of scope for an internal-app terminology pass.
- **"Services" → "Templates," extended to Workflow A's own surface.** §9.3 already established Templates as Workflow B's vocabulary for a bare `services` row. This round extended the same word to Workflow A's own `/services` pages (route, table, and every internal name unchanged — text only), plus the two pages genuinely shared by both workflows (a client's assigned-services section, `/collections`), which now render "Service" or "Template" based on the viewing organization's own `workflowType`. This also closed a real pre-existing gap: `/services/*` had no workflow gate at all, unlike `/templates/*`'s existing one, meaning a one-time organization could reach the Collection Day override field this same round hides everywhere else. `/services/*` now 404s for `one_time` organizations, symmetric with `/templates/*`.
- **"Audit Log" → "Activity History,"** with a real behavior change to match: the default view is now today's events (not the last 200 rows, unbounded by date), with Today/Last-7-Days/Last-30-Days/Custom-Range filters and day-grouped results. `listAuditLog`'s new `from` / `to` bounds are opt-in — every other caller (e.g. a Collection Request's own full history on `/collections/[id]`) is unaffected.
- **Two new small state flags**, both following this document's established idiom (a boolean on the owning row for a permanent fact, a nullable timestamp on `organizations` for "has this

one-time thing happened yet"): `services.isSampleTemplate` (true only for the templates `seedExampleTemplates` auto-creates, never for a user's own library addition or duplicate — see §9.3) powers a "Sample Template" badge and a short explanation banner; `organizations.sampleTemplateCardShownAt` gates a one-time dashboard promo card to exactly one appearance, the organization's first visit.

- **A complete password-reset flow** (`password_reset_tokens`) was added alongside login UX polish (show/hide password, a dual-duration "Remember Me," a real `/register` URL). The reset-email step is mocked per this document's established pattern for pilot-stage external dependencies with no live provider configured (Chapter 6's classifiers are the same shape of stand-in) — the reset link is logged server-side, never exposed in any HTTP response, and the flow never reveals whether a given email is registered.

None of this round touched Chapters 1–9's actual product logic — every change here is a label, a default filter, a small new flag, or closing a routing gap the two-workflow model itself already implied should exist.

Chapter 10 — Recurring vs. On-Demand: A Per-Item Choice, Not a Permanent Lock

Chapter 9 established `organizations.workflowType` as a permanent, set-once fork between two entire product experiences. Product Evolution M9 changed the underlying business model: **every organization can use both Recurring Collection and On-Demand Collection at once**, chosen per item (per Service), not once per organization. This chapter records what changed and, just as importantly, what Chapters 1–9 already got right and did not need to change.

10.1 The Real Gate Moved from the Organization to the Service

`services.collectionMode` (`"recurring" | "on_demand"`) is now the one predicate every runtime behavior checks — not `organizations.workflowType`. A Recurring Service (what Chapter 9 called a Business Type) automatically schedules its own next cycle and learns from its clients exactly as Chapters 1–8 describe; an On-Demand Service (what Chapter 9 called a Template) never does either, exactly as §9.3 describes — that distinction is unchanged. What changed is scope: both kinds of Service can now exist side by side in one organization, so the three functions that ever write learned state (`recordAdHocDocumentObservation`, `detectMissingRequirements`, `recordLearnedDocumentPattern`) resolve the specific Service a request belongs to and check *its* `collectionMode`, not the organization's. `organizations.workflowType` survives only as a record of which onboarding path the organization started on — it personalizes the wizard and picks a default dashboard, and nothing else reads it as a behavioral gate.

10.2 Onboarding: Three Choices, One Permanent Effect Removed

Onboarding's Collection Style step now offers three options — Recurring, On-Demand, or Both — matching the terminology used everywhere else in the product: **איסוף מחזורי** (Recurring Collection) and **איסוף לפי צורך** (On-Demand Collection). "Both" runs the fuller Recurring setup path, since On-Demand needs no dedicated setup steps of its own — it's simply available from the navigation the moment onboarding completes. Unlike Chapter 9's original design, this choice is no longer permanent or exclusive: the navigation always shows both "איסוף מחזורי" and "איסוף לפי צורך" once onboarding is done, regardless of which the organization started with, and either kind of Service can be created at any time from its own "new" page.

10.3 Recurring Collection Actually Recurs Now

Chapter 9 (and the schema comment on `organizations.collectionDayOfMonth` before this milestone) was explicit that no scheduler auto-created Collection Requests — every cycle was opened by an employee, by hand. `src/lib/recurringScheduler.ts` closes that gap: a new `services.collectionFrequencyIntervalMonths` ("every X months" — one field covers monthly, quarterly, semi-annual, annual, and any custom interval at once, deliberately not a separate unit/enum) and a per-assignment `client_services.nextCollectionRunAt` drive a new cron pass (`runRecurringCycleCreation`, called from `runScheduledTasks` alongside the existing reminder/inactivity/Drive-retry passes) that opens the next cycle — snapshotting requirements and sending the initial request through the exact same `startConversation` a manual "send" already used — the moment a pairing comes due, then advances the schedule from where it was supposed to fire (not from "now"), so a late tick never drifts the cadence forward.

This is deliberately server-side, not a UI convention: the query that finds due work filters on `services.collectionMode = "recurring"` as a hard `WHERE` clause. An On-Demand Service is structurally ineligible — there is no code path by which one could be picked up, not a flag that a future caller could forget to check.

10.4 Frequency and Reminders Are Explicitly Different Concepts

Frequency (`collectionFrequencyIntervalMonths`, resolved against the existing per-Service anchor-day override) decides when the *next cycle* starts. **Reminders** (the five fields Chapter 9 already knew about — business hours/days, reminder interval, inactivity timeout) decide what happens *inside* an already-open cycle that's still waiting on documents. They were easy to conflate before this milestone because both lived on the same override card; they now have their own card (`ServiceFrequencyCard` / `FrequencyField`) sitting beside the existing `ServiceScheduleOverrideCard`, and neither onboarding's Reminder Rules step nor a Service's own settings page treats them as one setting.

10.5 AI Suggests, the User Approves — Classification Is Never Silently Authoritative

Before this milestone, an automatic classification (import-time or a same-name guess on manual client creation) was written and made effective in the same step — `businessTypeId` was set *and* the

client was immediately linked to that type's Service, before any human had seen it. `clients.classificationConfirmedAt` now separates those two moments: an automatic classifier writes a *suggestion* (`businessTypeId` / `businessTypeConfidence` visible in the UI) but never creates the `clientServices` link and never records a learned synonym until a human confirms it — either by correcting it (a manual choice is confirmed by construction) or by approving it as-is (Step 5's per-type "Confirm" button, `confirmPendingClassifications`, approving every pending suggestion for one type in one click rather than one client at a time). Until confirmed, a suggestion cannot affect what's requested from that client — this is the concrete, code-level form of this document's own trust principle (Chapter 1: "Centro does not guess. It observes, suggests, confirms, and only then learns.") applied to classification, not only document requirements.

When a spreadsheet carries no usable type signal at all, Centro still never guesses — Step 5's existing unclassified-client picker gained a text filter and a select-all-filtered control, so dividing hundreds of clients into groups by hand stays fast rather than becoming a one-at-a-time chore.

10.6 WhatsApp and Google Drive Are Mandatory, Verified, and Recoverable

`tryActivateAutomation`'s BR-001 gate (Ch.3, extended in the Google Drive OAuth Integration round) used to be best-effort — onboarding could complete with neither integration connected. `finishOnboarding` now hard-blocks: `checkIntegrationStatus` (`src/lib/integrationRequirements.ts`) verifies WhatsApp's identifiers are present and actually attempts a Google token refresh (catching a revoked/expired connection, not just "was connected once") before onboarding can complete, sending the office back to the Connect step with a specific, honest reason otherwise. The same check gates the real moment a collection *starts* (`initiateConversation`), surfacing the exact required wording ("לא ניתן להתחיל את האיסוף... יש לחבר מחדש...") with a direct link back to Settings — which, for the first time, can also reconnect or disconnect both integrations after onboarding, not only during it.

10.7 A Document Is Never Lost to a Drive Outage

`uploadDocumentResiliently` used to clear a document's temporarily-held bytes (`documents.pendingFileContent`) the moment it was marked "approved," before knowing whether the Drive upload actually succeeded — a failure right after approval meant the real file was gone for good, leaving only a placeholder. The held bytes are now cleared only on confirmed success; a failure instead records `driveUploadFailedAt` and increments `driveUploadRetryCount`, and a new retry pass (`retryFailedDriveUploads`, called every cron tick alongside the recurring-cycle pass) re-attempts it automatically through the same resilient path. After `DRIVE_RETRY_MAX_ATTEMPTS` (8) consecutive failures, the document is marked `driveRetryExhaustedAt` and escalated as a critical audit event rather than retried forever — still holding its bytes, still recoverable by hand, never silently dropped.

10.8 Disable Automation Now Actually Disables It

The Settings "Deactivate automation" toggle already behaved like a live on/off switch in the database (`organizations.automationActivatedAt` set and cleared on demand) — it just wasn't consulted by the send pipeline. `sendOutboundMessage`, the one chokepoint every automated ("ai") message goes through, now checks it before doing anything else, so deactivating genuinely stops every automatic send and every automatic cycle-creation attempt org-wide, immediately, with no historical data touched. A narrower sibling, `services.automationPausedAt`, does the same for one Recurring Service at a time, leaving every other Service unaffected — both controls carry a plain-language "what does this do, will anything be deleted, will clients be messaged" explanation next to the button itself, not just in this document.

Chapter 11 — Smart Profession-Aware Onboarding

Centro's onboarding wizard was originally built accounting-firm-first. This chapter records a read-audit-first redesign that closed the specific gaps between that origin and the multi-profession product Centro has become — deliberately not a rebuild, since the audit found most of the wizard (starter document suggestions per business category, manual Template/Business Type CRUD, the AI-suggests/human-confirms classification gate) was already generic. What follows is what was actually missing.

11.1 A Classifier With No Opinion About What "Its Own" Candidates Are

`classifyClientBusinessType` (`src/lib/ai/businessTypeClassifier.ts`) matches imported row text against `BUSINESS_TYPE_SYNONYMS`, a dictionary of Israeli accounting terms (`עוסק מורשה`, `בע"מ`, …), unconditionally — it has no parameter for "which profession is this organization." Before this chapter, that was safe only by construction: an insurance organization's own `candidates` list (seeded per its `businessCategory`, Ch.10 §10.5/M2) never contains an accounting `canonicalKey`, so a dictionary hit for "עוסק מורשה" always fell through to a plain, unexplained "unclassified." The classifier now reports this case explicitly — `BusinessTypeClassification.conflict` — instead of silently discarding the signal. `importParsedRows` collects these into `ImportAnalysisSummary.conflicts` and Step 5 shows them by name: which client, what text matched, which (out-of-profession) type it resembles. Nothing is auto-applied from a conflict; it exists purely so the office can look, not so Centro can guess harder.

11.2 Templates Without Excel or AI

The manual Template/Business Type CRUD at `/templates` and `/services` already let an office build a checklist by hand — Chapter 9/10's own text is explicit that this was never Excel-only. What onboarding itself never offered was a way to reach that same CRUD *from inside the wizard*, without first uploading a file. `createManualTemplate` (`src/app/onboarding/actions.ts`) is a thin wrapper over the same `createBusinessType` call the classifier's own inline "+ new type" affordance already used — deliberately no `seedRequirements`, since the entire point is bypassing AI-suggested

defaults, not receiving a different set of them. It's offered as a persistent, secondary action on Steps 4 and 5 (`ManualTemplateCreator`), never a fork that replaces the Excel/AI path — an office can mix both in the same session, and every later step (Documents, Reminders, Summary) already treated all `businessTypes` rows identically regardless of origin, so nothing downstream needed to change.

11.3 One Bulk-Assignment Component, Not Two

Because a manually-created template is just another row in the same `businessTypes` list the classifier's suggestions already populate, Step 5's existing bulk filter/select-all/assign panel needed no new *logic* to cover both — only extraction into its own component (`ClientAssignmentPanel`) so the interaction (search a long client list, select-all-filtered, assign, or leave unassigned by simply not selecting) is defined once rather than copied if a future screen outside onboarding needs the same pattern.

11.4 Excel Rows Get One More Look Before They're `clients`

Column-mapping confirmation already existed as a review step, but only for *how columns map to roles* — the parsed rows themselves went straight into `clients` the moment mapping was resolved, whether resolved automatically or by hand. `importAndClassifyClients` and `confirmImportMapping` now both return a `needsRowReview` preview instead of writing; `confirmRowImport` is the only path that actually calls `importParsedRows`, after the office has seen every parsed row (name/phone/email/business-type column) and could deselect any of them. This is the concrete form of "Excel is context, never final truth" for row data, the same principle §10.5 already established for classification.

11.5 The Connect Step Becomes a Real Checkpoint

§10.6 described `finishOnboarding`'s hard-block check (`checkIntegrationStatus`) as the enforcement point for "both WhatsApp and Google Drive before automation can start" — true, but it was also the *only* enforcement point: nothing stopped a user from clicking through Steps 6–10 with neither connected, discovering the requirement only at Step 11's final submit, then being bounced all the way back to re-answer nothing (every prior step's data survives) but re-navigate the whole wizard regardless. `Step3Connect` (Step 5) now computes the same readiness (`driveReady` / `whatsappReady`) the finishOnboarding check does and simply doesn't render a working "Continue" until both are true, with a plain-language reason shown inline. No client-side polling was needed: every path that changes connection state (the Google OAuth callback, the WhatsApp Embedded Signup popup's completion handler, folder pick/create, disconnect) already forced a fresh server render of this exact page before this chapter, so the same server-computed `bothReady` value the button now gates on was already correct on every render — the change is entirely in what the button does with it. `finishOnboarding`'s own check is untouched and still runs — this closes the UX gap, not the only real enforcement, which remains server-side per this document's standing rule that client-side state is never trusted alone.

11.6 Staying Oriented: Which Profession Is This, Again?

`organizations.businessCategory` was chosen once, at Step 3, and never referenced again by any later step or by `WizardShell` — a real, if minor, way to lose track of what's being configured deep into an eleven-step wizard. `WizardShell` now accepts an optional profession badge (label + icon, `src/lib/businessCategories.ts`), shown from Step 4 onward — never at Step 3 itself, which is where it's still being chosen.

11.7 Final QA Pass — What a Code-Only Audit Missed

Sections 11.1–11.6 shipped first; a dedicated end-to-end pass (reading actual rendered screens, not just source) then caught what a code audit alone hadn't:

- **A live contradiction.** Step 5's own help text still read "you can connect these services later too" — true before §11.5, false after it. Rewritten to state plainly that both connections are required to continue, with the same "reconnect later via Settings" idea kept (that part stayed true) but no longer implying the *initial* connection was optional.
- **Two more accounting-only defaults**, both invisible to a grep for the classifier's own dictionary terms because neither is a classification concern: Step 4's own "what does Recurring mean" tooltip illustrated with "invoices, bank statements, payslips" regardless of the profession already chosen at Step 3; `OfficeInfoForm`'s business-name field (Step 2 — *before* profession is even chosen) suggested "Cohen & Co. Accounting Firm." Both are now profession-neutral.
- **A terminology split introduced by §11.2/§11.3 themselves**: the new `ManualTemplateCreator` / `ClientAssignmentPanel` said "תבנית" ("template") while every surrounding Step 4-8 screen already said "סוג עסק" ("business type") for the exact same `businessTypes` row — two names for one thing on one screen. Reworded to match the screens they actually appear on, not the spec language that inspired them.
- **A hand-rolled disabled button.** §11.5's disabled Continue button set its own `opacity-50` / `cursor-not-allowed` instead of relying on `buttonVariants`' existing `disabled:opacity-60 disabled:pointer-events-none` — invisible as a bug (it still looked disabled) but a real inconsistency with every other disabled control in the app, now removed in favor of the shared pattern.
- **A hand-rolled badge and a stale page reference.** The profession badge used a one-off `<span>` instead of the existing `Badge` component; Step 6 (Documents)'s empty state was a bare paragraph instead of the shared `EmptyState` component used by its neighbors (Step 5, Step 7) *and* pointed users to "the templates page" for a Recurring-only screen — the actual page is Services ("איסוף מחזורי" in the sidebar). All four fixed.

No new behavior in this pass — every fix is either a text correction or a swap to an already-existing shared component. Verified live again afterward: all three flows end-to-end, plus a dedicated check of a Recurring org with zero business types to confirm the corrected empty state.

Document History

Date	Change
Pilot Edition	Original 7 chapters established as the architectural source of truth.
Milestone 1	Migrated to a version-controlled Markdown source in-repo. Chapter 6 revised to state the Decision Hierarchy as actually implemented (Learned Knowledge checked before generic Business Rules) — see the Centro Implementation Report's Milestone 1 addendum for the full rationale. Added Chapter 8 (The Boundary of Learning) as a permanent constraint, in response to an early roadmap draft that would have violated it.
Milestone 2	Chapter 1/2's "every client begins in Learning Mode" implemented as a real, visible flag (`clients.learningMode`), ending the first time a client's first Collection Request reaches `completed` . No timing or duration data is captured — see Chapter 8; this milestone exists solely to gate Milestone 6's "document appears to no longer be needed" detection, which must never fire before a client has completed even one full cycle.
Milestone 3	Chapter 6's Decision Hierarchy generalized to document classification, its second real implementation (after Epic 4's business-type classifier). A client's own manually-corrected document-naming history (`document_learned_patterns` , scoped per client) is now checked before the generic deterministic heuristic, which now sits ahead of a documented AI-fallback stub — full parity with the 4-layer pattern.
Milestone 4	Chapter 4/7's "the accountant reviews exceptions, not routine learning" given a real, unified surface: the dashboard's existing per-feature queue pattern extended with a "suggested classification" view (clients auto-classified in the 70-94% band), sitting alongside the pre-existing document-review queue rather than a separate, undiscoverable screen.
Milestone 5	Chapter 3's "Confirm with the client, through WhatsApp" — the one lifecycle step that didn't exist anywhere yet — built as reusable, domain-agnostic infrastructure (`pending_confirmations`), with a real (not throwaway) manual trigger: an employee can ask a client to confirm a document becomes a standing part of their collection at the exact moment of manual assignment. A real bug was found and fixed during this milestone's own verification — see the Implementation Report's Milestone 5 addendum.
Milestone 6	Chapter 8's one permitted kind of learning, made real end to end: `client_document_requirements` tracks per-client additions and removals, confirmed only through Milestone 5's WhatsApp loop, applied only to that one client's future snapshots — the service template itself is never touched. Additions require a genuine second occurrence (never a single event); removals require two consecutive completed cycles missing the same document. A structural constraint (a cycle can never complete while a requirement is unsatisfied) required a new, independently

Date	Change
	useful capability — waiving a requirement for one cycle — to make recurring-absence detection possible at all; documented in the Implementation Report's Milestone 6 addendum along with a second bug (ambiguous reply resolution when two confirmations are open at once) found and fixed during verification.
Milestone 7	Pilot Hardening — no new product behaviour; a dedicated pass over everything M1–M6 built, with an emphasis on the organization-isolation boundary this document has assumed throughout but never itself audited end to end. Two real cross-tenant authorization gaps were found and fixed (a pending-confirmation resolution action, and a document-requirement-assignment action, both trusting a caller-supplied id without confirming it belonged to the caller's own organization) — see the Implementation Report's Milestone 7 addendum for the full detail, including a two-tenant Playwright test added specifically to keep this boundary verified going forward, not just asserted in prose. The Milestone 3-deferred `isFuzzyDuplicate` doc-comment discrepancy was also resolved (the comment's own example didn't clear its own threshold — corrected to a real, passing example; the function's deliberately strict behavior was left unchanged).
Product Evolution M1	The start of a separate roadmap generalizing Centro from an accounting-only platform into a document-collection platform for any business (see the Product Evolution milestone roadmap). This milestone captures two new, purely onboarding-time facts and changes no existing behaviour: which kind of business the office itself is (`organizations.businessCategory` — distinct from `business_types` / `clients.businessTypeId` , which classify this office's *clients* and remain exactly as Chapters 6 and 8 describe), and which of two permanent workflows it operates in (`organizations.workflowType` : `recurring` , today's exact learning-driven experience, or `one_time` , a template-driven experience with deliberately no learning at all, built out in later milestones). Every existing organization defaults to `accountant` / `recurring` , so nothing in Chapters 1–8 changes for any organization that existed before this milestone. The one-time workflow's own governing chapter will be added once it's real (the Product Evolution roadmap's final hardening milestone).
Product Evolution M2	Generalizes Workflow A's onboarding *starting point* — never the recurring learning engine itself — per the office's declared business category. For `accountant` / `tax_advisor` , the starter client-types and suggested documents are byte-identical to before this milestone. For every other declared category, a new mocked-AI module (`src/lib/ai/businessCategorySuggestions.ts` , same documented "real interface, no live provider configured" pattern as every other AI module in this codebase) supplies curated, category-specific starter types — and, for a genuinely novel free-text "Other" category, a broad keyword-matching layer covering ~20 common business domains before falling back to a starting point named after the office's own words, never a generic or empty placeholder. Once seeded, a business type behaves identically regardless of which tier produced it: Chapter 3's Observe → Suggest → Confirm → Learn lifecycle, Chapter 6's Decision Hierarchy, and the entire recurring learning engine apply exactly as before — this milestone only ever changes

Date	Change
	what a client-type's document checklist starts as, never how Centro learns from that point on.
Product Evolution M3	The onboarding wizard genuinely forks for the first time. Steps 1–5 (Welcome through Connect) stay identical for both workflows; from Step 6 onward, `organizations.workflowType` determines a completely different continuation. Workflow A's Steps 6–11 are untouched (regression-verified). Workflow B gets its own, deliberately shorter path (9 steps total): an explicitly optional Client Import step that stores *only* name and phone — no classification, no learned-synonym writes, no `business_types` seeding, no analysis summary, honoring the product's "Centro never learns" boundary for this workflow from the very first cycle, not just later once Templates exist — followed by a Working Hours step (business days/hours only, deliberately never asking about reminder cadence or collection-day-of-month, which remain office-policy concepts with no meaning for a one-off request), then an adapted Summary and Completion. `WizardShell`'s step count and titles are now resolved per request instead of fixed constants, the deferred piece of work flagged at the end of Milestone 1. A full live walk of both paths (recurring regression and both one-time branches) passed cleanly on the first run — the step-renumbering bug class Milestones 1 and 2 both found instances of had already been fully swept by Milestone 2's codebase-wide check.
Product Evolution M4	One-time-workflow organizations get their own dashboard and navigation, sharing the underlying route and shell rather than a separate layout — matching the spec's own "Shared Infrastructure" list, which names Dashboard Infrastructure as shared. `/dashboard` branches at the top by `organizations.workflowType`; the sidebar's nav list does the same. The one-time dashboard reuses the recurring dashboard's *pattern* (small, independent, composable query functions) but none of its queue vocabulary, since business-type suggestions, pending classification confirmations, and the rest are Workflow-A-only concepts. A minimal `/templates` placeholder, guarded to one-time organizations only, keeps the new "תבניות" nav link and dashboard CTA from being dead links until Milestone 5 builds the real Templates CRUD. A recurring organization's dashboard and navigation are byte-identical to before this milestone, confirmed via live regression.
Product Evolution M5	The real Templates CRUD, replacing Milestone 4's placeholder — create/edit/delete/duplicate a Template, and add/rename/reorder/remove its document requirements, all as thin Template-branded wrappers around the exact operations `/services` already had (a Template *is* a bare `services` row). Reordering is simple move-up/move-down rather than drag-and-drop, deliberately — new `service_document_requirements.position` column, null for every pre-existing row and for the recurring workflow generally, so ordering falls back to creation order with no special-case code. The Template Library (`suggestTemplateLibrary` , extending Milestone 2's AI-suggestions module) offers one-click starter templates per business category, including a dedicated set for `accountant` / `tax_advisor` distinct from Workflow A's client-classification starters — a one-time office's request templates and a recurring office's client-type taxonomy are different concepts even for the same

Date	Change
	declared category. The first visit to `/templates` auto-seeds three Library suggestions as real, immediately editable templates, mirroring `seedStarterBusinessTypes`'s own idempotent-on-emptiness pattern.
Product Evolution M6	Client assignment for Templates — reuses `client_services` directly (the same join table the existing client-detail-page "assign service" action already writes) rather than a new table, since "clients assigned to this template" and "clients assigned to this service" are the same relationship once a Template is understood as a bare `services` row. The Template detail page gained an assignment panel mirroring the onboarding wizard's own unclassified-clients picker (a checkbox list + submit), plus an inline "create new client" shortcut for the common one-time-workflow case of assigning a template to someone not in the system yet. A client can be assigned to any number of templates simultaneously, and the assignment panel deliberately stays open after a successful assignment so several clients can be added in one sitting.
Product Evolution M7	Send Request — the step that actually delivers a Template to its assigned clients, closing the loop into the existing collection pipeline. One submit creates a `collectionRequest` per selected client (reusing `snapshotServiceRequirements`, unchanged) and either sends immediately or schedules a future delivery — deliberately one mechanism, not two: a new nullable `collection_requests.scheduledAt` is null for "send now" (delivered synchronously) and non-null for "schedule for later," and the exact same function, `attemptScheduledDelivery`, delivers both — the only difference is who calls it and when. The cron tick (`runScheduledTasks`) gained a due-scheduled-request query so a "send now" that lands outside business hours degrades gracefully into "delivered on the next tick" rather than silently failing, using infrastructure that already existed for reminders. Chapter 8's learning boundary is now enforced at the three functions that ever write learned state (`recordAdHocDocumentObservation`, `detectMissingRequirements`, `recordLearnedDocumentPattern`), not only by omission in Workflow B's UI — a new `isOneTimeWorkflowOrganization` check is the first line of each, so the guarantee holds even if a future caller forgets to check first. Verified live, including the full scheduled-delivery path (a request scheduled ~65s out was confirmed to stay `draft` until its time arrived, then flip to `active` via a real cron-tick run) and the learning guard (the exact two-occurrence condition that triggers a client-confirmation prompt for a recurring client was reproduced on a one-time client and confirmed to never surface it).
Product Evolution M8	Pilot Hardening for the whole epic — added Chapter 9 (The Two-Workflow Product Model) and extended Chapter 8 with Workflow B's absolute "never observed" form of the learning boundary; no prior chapter's content changed. A cross-tenant Playwright audit (two independent one-time organizations) confirmed every new M1–M7 surface — `/templates`, `/templates/[id]`, the one-time dashboard, `sendTemplateRequest`, and every Template CRUD/assignment action — correctly rejects or hides another organization's data, with zero new isolation gaps found; every action was already scoping by `session.organizationId`, a direct consequence of M5–M7 building each one as a thin wrapper around the pre-existing, already-audited `services` / `clients`

Date	Change
	data-access functions rather than new parallel ones. One genuine gap *was* found and fixed: nothing previously stopped a fully onboarded organization from navigating straight back to `/onboarding` and resubmitting `updateWorkflowType`, silently flipping a live organization between the recurring and one-time workflows post-onboarding — contradicting this epic's own "permanent, set-once" design (§9.1). Fixed with one guard, mirroring the `(app)` layout's existing reverse-direction check: once `onboardingCompletedAt` is set, `/onboarding` itself redirects to `/dashboard` before any onboarding action can run. A second, smaller gap — four of `sendTemplateRequest`'s rejection paths (invalid schedule, no clients selected, etc.) redirected with an `?error=` code the template page never actually displayed, leaving the user with silent, unexplained failures — was also found and fixed. Additional edge cases verified without incident: a ~1,000-character free-text business category produces sensible, non-crashing Template Library suggestions; a template with zero assigned clients simply hides the Send Request panel rather than presenting a dead-end form.
UX Polish M1–M8	A dedicated UX-only pass (Chapter 9, §9.5) — no architecture or business-logic change. Terminology aligned with the two-workflow model ("Office"→"Business," "Services"→"Templates" for Workflow A's own pages, "Audit Log"→"Activity History" with a real default-to-today rewrite), plus a `/services/*` workflow gate closing a pre-existing gap symmetric with `/templates/*`'s own. New `services.isSampleTemplate` / `organizations.sampleTemplateCardShownAt` flags power a one-time "Sample Template" onboarding nudge. A complete password-reset flow (`password_reset_tokens`) shipped alongside login UX polish (show/hide password, a dual-duration "Remember Me," a real `/register` URL). One real, pre-existing bug was found and fixed during this round's own regression pass: `TemplateSendRequest`'s client-selection state was seeded once from `useState`'s initializer and never updated when a client was assigned *after* the component first rendered, leaving "Send" permanently disabled at 0 selected until a full page reload — fixed by adjusting the selection during render when the `assignedClients` prop actually changes (React's own documented pattern for this case), not via a `useEffect`.
UI Redesign M0–M9	A full visual and interaction redesign of the authenticated application (login through every internal screen), executed against the Centro Design Manifesto v1.0. Zero product-logic change — no schema, API, workflow, or business-rule modification; every chapter above remains exactly as implemented. Purely additive Tailwind design tokens (`src/app/globals.css`), a new shared component library (`src/components/app/`: `Table`, `Tabs`, `KpiCard`, `AuthCard`, `ConfirmDialog`, `DevToolsPanel`, `ProgressBar`, `AiBriefing`, an extended `FormField`), and a per-screen visual migration across the app shell, onboarding wizard, both dashboards, clients, services/templates, collections, settings, and activity history — the latter two's zero-client-JS, bookmarkable-URL filter mechanisms preserved exactly, restyled only. The public marketing Landing Page was explicitly out of scope and confirmed pixel-identical throughout (zero diff on `src/components/landing/**` across all ten milestones). Two real cross-cutting bugs were found and fixed during this epic's own

Date	Change
	verification, both documented in the redesign's own final report: `ConfirmDialog` 's original `cloneElement` -based trigger API broke intermittently under React Server Components (fixed by accepting plain `ReactNode` content instead of an element to clone), and this document's own Architecture PDF (`reports/Centro-Architecture-and-Philosophy.pdf`) was discovered to be a broken artifact — a printed "file not found" browser error page, not the document's actual content — regenerated correctly as part of this pass.
UI Redesign — Centro Glow Phase 2	A second, iterative visual round on top of M0–M9's redesign, refining it into the "Centro Glow" language: near-white surfaces with extremely subtle per-card hover glow (behind the card, color-matched to its own icon, never inside it), soft gradient icon badges, and a small "Centro Active" status indicator on the Dashboard only (a breathing dot + tooltip, aligned to the page title via real flex layout, not a guessed offset). The onboarding wizard got the most visible changes: the official `CentroMark` brand mark now appears at the top of every step and on Login/Register (glow/breathe wrapper only — the mark's own SVG geometry is never touched, a permanent rule); the Welcome step's animated hand was removed entirely in favor of a staggered typography-and-motion reveal (glow → eyebrow → blur-to-sharp title → subtitle → rising button) after review found every CSS/SVG attempt at a hand read as artificial; Service Connections shows real Google Drive/WhatsApp brand marks and a shared circle-draws-into-checkmark animation (`AnimatedCheckBadge`); the AI-analysis step reveals progressively (title, card, text, then each stat counting up) instead of appearing instantly; Setup Summary rows animate in with the same checkmark draw, staggered; and onboarding completion replaced a static particle badge with a real canvas confetti burst (blue/purple/teal/white/gold, ~1.7s). Zero product-logic change — every refinement is visual/motion only, built from static mockups reviewed and explicitly approved screen-by-screen before implementation. One real bug was found and fixed during this round's own live verification: two CSS classes applied to the same Setup Summary row both set the `animation` shorthand, and the later rule silently discarded the earlier one's `animation-name` entirely (equal specificity) — every "done" row was stuck invisible until the two animations were merged into one shorthand declaration.
Product Fixes & Mobile Responsiveness	A focused round of product fixes plus a full-product mobile responsiveness audit, no visual redesign. **Fixes:** real, complete Hebrew Privacy Policy and Terms of Service content (replacing placeholder text), now linked from Login/Register and every onboarding step's footer, not only the registration checkboxes. The one-time workflow's client import was brought to parity with the recurring workflow's upload/replace/add-another controls — reusing the exact same `clients.importedDuringOnboarding` flag and replace-deletion query the recurring import already relies on (no schema change; the flag already existed as a generic column, just previously unset by `importClientsSimple`). The import step now stays in place after a successful upload to show a review state with Replace/Add-another and an explicit Continue, instead of auto-advancing — mirroring the self-redirect-to-the-same-step trick `importParsedRows` already used for the recurring workflow's own review step. Setup Summary rows now show a red X for an incomplete/skipped step instead of an empty hollow circle, in both workflows, row text unchanged. The one-

Date	Change
	time Dashboard's four KPI cards are now clickable (Templates, Clients, Collection Requests), reusing `KpiCard`'s existing `href` behavior — zero visual change. Internal reference codes in parentheses (e.g. `(BR-18.1)`, `(Ch.17)`) were removed from the user-facing Settings and Activity History descriptions. **Responsiveness:** a systematic audit across the landing page, every auth screen, the full onboarding wizard (both workflows), and every main app screen at five phone/tablet viewport widths found one real bug — the Welcome screen's ambient glow blob paired a logical Tailwind position class (`start-1/2`) with a physical transform (`-translate-x-1/2`); correct in LTR, but under this app's RTL layout the two no longer cancel out, and the blob was pushed far outside the viewport, forcing real horizontal overflow on every phone size (fixed with the matching physical `left-1/2`). Every other flagged page turned out to be a `scrollWidth` measurement artifact already neutralized by the existing `body { overflow-x: clip }` safeguard, confirmed via a direct `scrollTo`-based check rather than assumed.
Landing Page — Contact Form Email & WhatsApp Button	Two additions to the public landing page only; the app's authenticated surfaces and the landing page's own visual design were untouched. **Contact form → email:** `ContactForm.tsx` (shared by `ContactSection` and `DemoRequestModal`) previously simulated a network call and always "succeeded." It now posts to a new `POST /api/contact` route handler, which re-validates the payload server-side (never trusting client-side validation as a security boundary) and sends a formatted HTML+text email via Resend to a configurable inbox (`CONTACT_EMAIL_TO`, defaulting to `Centro.ai.team@gmail.com` in code) from a configurable sender (`RESEND_FROM_EMAIL`, defaulting to Resend's sandbox sender), including every submitted field and the submission timestamp. The success UI only ever renders after a real `2xx` from the route; any failure (missing `RESEND_API_KEY`, a Resend-side error, a network failure) surfaces as an inline error message and re-enables the submit button for retry, rather than a false-positive success. The submit button already guarded against duplicate submission via its `status === "submitting"` disabled state; a redundant top-of-handler guard was added for defense in depth. `RESEND_API_KEY` is read only in the route handler (server-side), never sent to the client. **Floating WhatsApp button:** a new `FloatingWhatsAppButton.tsx`, mounted only from `src/app/page.tsx` (never the root layout), so it appears exclusively on the landing page and never on any authenticated or auth-flow screen. Fixed bottom-right (physical `right-*`, not the logical `end-*` that would resolve to the opposite corner under this app's RTL layout — the same class of mistake Mobile Responsiveness's one real bug above was), respecting `env(safe-area-inset-*)`, positioned opposite `AccessibilityWidget`'s existing bottom-left placement so the two never collide. Reuses the existing `WhatsAppGlyph` icon and `--color-whatsapp` design token rather than introducing new brand colors, with one new additive keyframe (`centro-whatsapp-breathe`, a slow ambient glow pulse behind the button, echoing the app's established "glow behind, never inside" hover mechanic) — already covered by the existing global and manual `prefers-reduced-motion` overrides with no bespoke exception needed. Opens `https://wa.me/972559871812` with a pre-filled

Date	Change
	Hebrew message, `target="_blank" rel="noopener noreferrer"`, and a Hebrew `aria-label` for screen readers.
Google Drive OAuth Integration	Google Drive's "Connect" button (Step3Connect, onboarding step 5, shared by both workflows) is real for the first time — everywhere else in this document that still says "Google Drive is mocked" now refers only to `src/lib/storage/driveAdapter.ts`'s per-client folder creation and document upload, which remain an explicit, deliberate boundary of this round (see below). The button is now a plain link to `GET /api/auth/google/start`, which redirects to Google's real authorization endpoint requesting only `drive.file` — Centro can see and act on exactly the files/folders it creates itself, plus whatever the user explicitly grants through the Google Picker widget; it is structurally incapable of listing or reading the rest of a user's Drive. `access_type=offline` + `prompt=consent` guarantee a refresh token every time. A random `state` value round-trips through an httpOnly cookie for CSRF protection, checked by `GET /api/auth/google/callback` before any token exchange happens. Tokens are encrypted at rest with AES-256-GCM (`src/lib/googleAuth/tokenCipher.ts`, key from `GOOGLE_TOKEN_ENCRYPTION_KEY`) in two new `organizations` columns; a refresh-token response is never blindly trusted to include a new refresh token (Google only issues one on first-ever consent), so `storeTokens` preserves the existing one when a refresh omits it. `getValidAccessToken` (`src/lib/googleAuth/driveTokens.ts`) transparently refreshes an expiring token before handing it to a caller, whether that's a server-side Drive API call or the new `GET /api/google-drive/access-token` route (short-lived access token only, refresh token never leaves the server) that the client-side Google Picker needs to open. Once connected, the user chooses exactly one folder Centro is scoped to for that organization — an existing folder via Picker (the only way to grant `drive.file` access to something the app didn't create) or a newly created one via a plain server action (`createGoogleDriveFolder`, no client JS needed) — and a Picker-selected folder id is never trusted at face value: `selectGoogleDriveFolder` re-fetches it from the Drive API to confirm it's real, accessible, and actually a folder before storing it. `tryActivateAutomation`'s existing BR-001 gate (Ch.3) was extended to also require `googleDriveFolderId`, not just `googleConnectedAt` — automation cannot activate with an account connected but no folder chosen, since Centro would have nowhere to act. Disconnecting (`disconnectGoogleDrive`) revokes the token with Google (best-effort) and clears every stored credential and the selected folder.
Real Document Storage in Google Drive	Closes the gap the previous round explicitly flagged: `driveAdapter.ts`'s `ensureClientFolder` / `uploadDocument` now call the real Drive API instead of generating mock IDs, using the connected organization's own stored token and selected root folder — every write stays scoped to exactly that one folder, matching the `drive.file` grant's own promise. `ensureClientFolder` lazily creates a real subfolder per client (named after the client) nested inside the organization's root folder, storing the real folder id on `clients.driveFolderId` exactly as the pre-existing column already assumed; `uploadDocument` performs a real Drive multipart upload (`src/lib/googleAuth/drive.ts`'s `uploadDriveFile`, hand-built `multipart/related` body — no new dependency) and stores the real returned file id

Date	Change
	on `documents.googleDriveFileId`. Both functions throw a typed, catchable error (`GoogleNotConnectedError`) when an organization approves a document before ever connecting Drive, rather than a generic failure. **What "upload the file" means without a real inbound-document channel yet:** this product still has no real WhatsApp message receipt (mocked throughout `src/lib/conversationOrchestration.ts`), so there is no source of real file bytes for documents that arrive through the WhatsApp simulator — those uploads still go through, with a small, clearly-labeled Hebrew placeholder file explaining exactly that, rather than pretending to have content that doesn't exist. The one place in the product today with genuinely real bytes is a manual document upload by an employee — `addManualDocument` (`collections/actions.ts`) gained an optional real `<input type="file">`, and when a real file is attached, its real bytes and mime type flow straight through to the real upload, no placeholder involved. `uploadDocumentResiliently` (the existing retry/graceful-degradation wrapper) moved from `collections/actions.ts` into `driveAdapter.ts` itself and is now shared by all three approval call sites — manual add, employee review, and the WhatsApp simulator's AI auto-approval path, which previously called `uploadDocument` directly, unwrapped; a real Drive failure there would have crashed the whole inbound-message flow instead of degrading gracefully like the other two paths already did, a real latent bug this round fixed as a natural consequence of making the underlying call actually capable of failing for the first time. Verified live end-to-end against the one real, already-connected production organization: a genuine file (not a simulated one) was uploaded through the deployed app, and both the resulting Drive folder id and file id were confirmed directly against the Drive API to be real, accessible objects — not just plausible-looking strings — with the corresponding audit trail showing a clean success and zero failure/skip events.
Contact Form — Full Payload, Spam Protection, Server-Side Logging	Extends the previous round's basic Resend integration into the complete contact-form flow. `ContactForm.tsx` gained two new optional fields (business name, message) plus a `source` prop distinguishing the inline `ContactSection` from the popup `DemoRequestModal` — every field, plus the submission timestamp and source, is included in the email sent to `CONTACT_EMAIL_TO`. **Spam protection**, added without any external CAPTCHA service or dependency: an `sr-only` honeypot field (applied to the input itself, not just a wrapper — a real bug found and fixed during this round's own testing, since an ancestor's `sr-only` clips what's *painted* but not the child's own layout box, which a more sophisticated bot checking element geometry directly could still detect) that simple bots often auto-fill; a minimum-time-since-render check (a real visitor never reads and fills a form in under 1.5 seconds); and IP-based rate limiting (a small dedicated in-memory limiter in `route.ts` itself, same "single pilot instance" pattern and caveat as `src/lib/auth/rateLimiter.ts`, not the same module since the two need different windows/limits). Both spam checks deliberately return the same `200 OK` a real success would — showing a bot a distinguishing rejection only teaches it to adapt, and neither check can ever fire for a genuine visitor. **Server-side logging**: every stage (submission received, spam-blocked, rate-limited, sent successfully, or failed) now logs a structured line server-side, not just failures as

Date	Change
	before. Verified live: every new field renders and reaches the server correctly, the honeypot and timing checks return a success-shaped response without an email actually being sent, and real validation errors and rate-limiting still surface correctly. With a real Resend API key (provided by the account owner) configured in Vercel, a genuine end-to-end submission through the actual deployed `/api/contact` route — all fields, real network round-trip to Resend, real acceptance — was confirmed working, sent to the Resend account's own verified address. **One real, currently-open gap**: Resend's account is in sandbox/testing mode (verified against `noammaarachot@gmail.com`, not a verified sending domain), so it currently rejects sending to any other address — including the real target inbox, `Centro.ai.team@gmail.com` — with a clear `403`, which the route correctly surfaces as an honest failure rather than a false success. Fixing this permanently requires verifying `centro-ai.co.il` as a sending domain in Resend, which requires adding DNS records at the domain's registrar — external account access only the account owner has, since the domain's DNS is not Vercel-managed.
Contact Form — Gmail SMTP Replaces Resend	Resend required verifying `centro-ai.co.il` as a sending domain before it would deliver to any address other than the account's own signup email — this needed DNS changes (DKIM/SPF/MX records) at the domain's registrar (LiveDNS), whose panel turned out to silently reject nameserver-delegation changes through its records UI (the real control lives on a separate "Manage Domain" page, confirmed against LiveDNS's own documentation) — a real, external blocker with no code-side fix. Rather than keep the domain-verification dependency, the contact form (`POST /api/contact`) now sends via Gmail SMTP (Nodemailer, replacing the `resend` package) using an account's own App Password (`GMAIL_USER` / `GMAIL_APP_PASSWORD`) — no domain verification, no DNS records, no external DNS provider required at all. Every other part of the contact-form work from the previous two rounds (full field payload, honeypot + timing + rate-limit spam protection, server-side logging, validation) is unchanged; only the final send call was swapped. Verified live with a real submission through the actual deployed form, confirmed delivered. The Resend domain resource and its API keys were removed/cleaned up as no longer needed.
Real WhatsApp Integration — M-WA-1 (Schema)	The start of a five-milestone plan (approved in advance, not built ad hoc) to replace WhatsApp's 100%-mocked integration with a real one, across three goals: Centro auto-messaging every landing-page lead, each organization connecting its own WhatsApp Business Account, and that connection running the existing, already-real conversation/document-collection engine for real. Architecture decision: unlike Google Drive's necessarily per-organization OAuth tokens, WhatsApp uses the Meta Tech Provider model — one shared System User access token sends/receives for every connected organization's number, scoped per call by that organization's `phone_number_id` ; no per-org token to store, encrypt, or refresh. This milestone is schema only, purely additive, and changes no behavior: four new `organizations` columns (`whatsappBusinessAccountId` , `whatsappPhoneNumberId` , `whatsappDisplayPhoneNumber` , `whatsappVerifiedName` — the pre-existing `whatsappConnectedAt` is reused as-is) and a new, deliberately organization-unscoped `leads` table (Centro's own sales leads, not a customer organization's data) for the not-

Date	Change
	yet-built Feature 1 lead-welcome flow. Verified as a true no-op: every existing WhatsApp-touching flow (the mocked connect/disconnect buttons, the conversation engine, the scheduler) confirmed unchanged.
Real WhatsApp Integration — M-WA-2 (Real Per-Org Connect)	WhatsApp's "Connect" button (Step3Connect, onboarding step 5) is real for the first time, via Meta Embedded Signup rather than an OAuth redirect — the flow stays inside a popup the whole time (`WhatsAppConnectButton.tsx` loads the Facebook JS SDK and calls `FB.login()` with the app's Embedded Signup Configuration), so the result reaches Centro two ways at once: Meta posts a `WA_EMBEDDED_SIGNUP` `postMessage` (checked against Meta's own origin) carrying the new WABA id and phone_number_id, while `FB.login()` 's own callback returns a short-lived authorization `code` . Both are sent together to a new `POST /api/auth/whatsapp/callback` (a JSON-returning POST, not a redirect — Google's callback is a redirect only because its flow ends in a real page navigation back from accounts.google.com, which Embedded Signup never does). The route exchanges the code (`src/lib/whatsapp/embeddedSignup.ts` 's `exchangeSignupCode` — this confirms the signup completed on Meta's side; the resulting token itself is discarded, since Centro sends/receives through the one shared System User token, not a per-org one), subscribes Centro's app to that WABA's webhooks (`subscribeToWabaWebhooks` , required separately — Embedded Signup connects the number but doesn't implicitly wire up webhook delivery), and resolves the phone number's real display/verified name from the Graph API rather than trusting anything the client reported (`src/lib/whatsapp/phoneNumbers.ts` 's `getPhoneNumberDetails`) before storing the connection (`src/lib/whatsapp/wabaTokens.ts`). `WhatsAppConnectionRow` (replacing the old generic mock `ConnectionRow` for this one integration) shows the real connected number once linked, mirroring `GoogleDriveConnectionRow` 's existing precedent for "a connection that needs more than a plain form action." Disconnecting clears all four new columns plus `whatsappConnectedAt` . Every other consumer of `whatsappConnectedAt` — `tryActivateAutomation` 's BR-001 gate, Settings' `integrationsReady` , Step 8's Setup Summary — needed no changes, since the column they already read didn't move, only what writes it did. `tsc` /lint/build/tests all clean; live verification against a real Meta test WABA is pending the account owner's Meta App credentials (App ID/Secret, System User token, Embedded Signup Configuration ID).
Product Evolution M9 — Recurring vs. On-Demand Collection	See Chapter 10 for the full rationale. Summary: the permanent, organization-wide "recurring vs one-time" lock (Chapter 9) became a per-Service choice (`services.collectionMode`) so one organization can run both at once; onboarding's Collection Style step gained a third "Both" option; a real automatic cycle-creation engine (`src/lib/recurringScheduler.ts`) was built from scratch (`services.collectionFrequencyIntervalMonths` + `client_services.nextCollectionRunAt` , driven by a new cron pass), closing a gap this document had explicitly flagged since Chapter 9 — On-Demand Services remain structurally unable to auto-recur (a hard `WHERE collectionMode = 'recurring'` clause, not a convention); frequency and reminder cadence are now modeled and surfaced as explicitly separate settings; automatic classification is a suggestion only

Date	Change
	(`clients.classificationConfirmedAt`) until a human confirms it, closing a real trust gap where a guess could become authoritative before anyone saw it; WhatsApp and Google Drive became a hard-blocking, live-verified onboarding requirement (`src/lib/integrationRequirements.ts`) instead of best-effort, with a Settings-based reconnect path added since one now genuinely needs to exist; a Drive upload failure right after approval no longer risks losing the document (bytes are held and automatically retried, up to 8 attempts, before escalating); and the "Deactivate automation" toggle, which already behaved like a live switch in the database, is now actually consulted by the send/scheduling pipeline, joined by a narrower per-Service pause. Verified live end-to-end: full registration → onboarding → hard-block-without-integrations flow; classification-suggestion → confirm flow (pending count provably decreasing per confirmation); industry-specific document personalization (a business-consultant category correctly never offered the accounting-specific starter types, and vice versa); pause/resume on a live Service; the global automation toggle; a live-triggered blocked-send showing the exact required reconnect message and a working Settings link; and, most importantly, the automatic engine itself against a real due schedule — a cycle was created for the recurring pairing, the schedule advanced (confirmed non-repeating on an immediate second run), and zero cycles were ever created for the on-demand pairing across the same run.
Smart Profession-Aware Onboarding	See Chapter 11. A read-audit-first pass (not a rebuild — most of the wizard was already profession-generic) that closed six specific gaps: the classifier now reports a cross-profession "conflict" instead of silently discarding an out-of-profession dictionary hit; onboarding gained a persistent "create your own template" path reusing the existing manual Template/Business Type CRUD, never a fork replacing the Excel/AI path; the bulk client-assignment interaction was extracted into one shared component (`ClientAssignmentPanel`) instead of building a second one; Excel-imported rows now get a full row-level review-and-deselect screen before a single row is written to `clients` , closing the one part of "Excel is context, never final truth" the classification-suggestion gate didn't already cover; the Connect step (Step 5) itself now withholds "Continue" until both WhatsApp and Google Drive are verified, instead of the only enforcement being Step 11's final bounce-back; and the chosen profession stays visible in the wizard header from Step 4 onward. No database migration — every change reuses existing schema (`businessTypes` , `clientServices` , `organizations.businessCategory`).
Smart Profession-Aware Onboarding — Final QA Pass	See Chapter 11 §11.7. A dedicated end-to-end review (reading rendered screens, not just source) before closing the project, catching what the original code-only pass missed: Step 5's help text still said connecting was optional, directly contradicting the new hard checkpoint; Step 4's collection-style tooltip and `OfficeInfoForm` 's placeholder both defaulted to accounting examples regardless of profession; the new manual-template UI said "template" while every surrounding screen already said "business type" for the same entity; the disabled Continue button and the profession badge each used one-off styling instead of the app's existing disabled-button and Badge patterns; Step 6's empty state was a bare paragraph pointing to the wrong page

Date	Change
	name. All text/consistency fixes, no new behavior — re-verified live across all three flows afterward.